0. @Retryable 과 RetryTemplate

Spring Framework 7.0부터 코어에 재시도 기능이 들어왔다. 이전에는 별도 프로젝트인 Spring Retry를 의존성으로 추가해야 했지만, 이제는 `spring-core`와 `spring-context`만으로 재시도를 쓸 수 있다. 방법은 두 가지다. 어노테이션으로 선언하는 `@Retryable`과 코드로 직접 실행하는 `RetryTemplate`이다. 이번 글에서는 두 방식의 차이와 언제 무엇을 써야 하는지 정리한다.

---

0. @Retryable과 RetryTemplate

- **`@Retryable`** : "이 메소드 호출 전체를 재시도하라"를 **어노테이션으로 선언**한다. `ApplicationContext`에 등록된 빈의 메소드에 사용한다.
- **`RetryTemplate`** : "이 코드 블록을 재시도하라"를 **코드로 직접 실행**한다.

> 이 둘은 같은 엔진을 사용한다. 명령형(imperative) `@Retryable` 메소드는 해당 `RetryPolicy`를 가진 `RetryTemplate`을 통해 호출된다.

구분	`@Retryable`	`RetryTemplate`
패키지	`org.springframework.resilience.annotation`	`org.springframework.core.retry`
방식	선언형 (AOP 프록시)	프로그래밍 방식
재시도 단위	메소드 호출 전체	감싼 코드 블록
적용 대상	Spring Bean의 메소드	어디서나

구분	`@Retryable`	`RetryTemplate`
소진 시 예외	마지막 원본 예외	`invoke()` : 마지막 원본 예외 `execute()` : `RetryException`(checked)

기본값

두 방식 모두 기본 정책이 같다. **모든 예외에 대해, 최초 실패 이후 최대 3번 재시도하고, 재시도 사이에 1초를 기다린다.**

> **주의** : `maxRetries`는 **재시도 횟수**다. 최초 호출은 포함하지 않는다. 총 시도 횟수 = 최초 1회 + `maxRetries`회 즉, `maxRetries = 4`면 최소 1번, 최대 5번 호출된다.

Spring Retry 라이브러리의 `@Retryable(maxAttempts = 3)`은 최초 호출을 **포함한** 횟수였다. 이름이 같은 어노테이션이라 마이그레이션할 때 헷갈리기 쉽다.

0.1 @Retryable

- **선언형**이며 메소드 호출 전체를 재시도 단위로 둔다. AOP 프록시 기반이라 **프록시를 거친 호출만** 재시도된다. (같은 클래스 안에서 `this.method()`로 부르는 자기 호출은 적용되지 않는다)
- **Spring Bean의 메소드**가 적용 대상이다. 메소드 레벨에 붙이면 해당 메소드만, 타입 레벨에 붙이면 그 클래스 계층의 프록시 호출 메소드 전체에 적용된다.
- 활성화하려면 `@EnableResilientMethods`를 선언하거나 `RetryAnnotationBeanPostProcessor` 빈을 등록해야 한다. 붙이지 않으면 어노테이션은 그냥 무시된다.

- `Mono` / `Flux`**를 지원한다.** 리액티브 반환 타입이면 RetryTemplate이 아니라 Reactor의 retry 기능으로 파이프라인을 감싼다.
- 재시도가 소진되면 **마지막 원본 예외**를 그대로 던진다. 중간에 발생한 예외들은 `MethodRetryEvent`로 발행되므로, 이벤트 리스너로 추적할 수 있다.
- 동작을 테스트하려면 프록시가 필요하므로 **Spring Context가 필요하다.**

```
@Configuration
@EnableResilientMethods
class AppConfig {
}
```

```
@Service
public class NotificationService {

    // 기본값: 모든 예외, 최대 3번 재시도, 1초 간격
    @Retryable
    public void send() { ... }

    // 특정 예외만 + 지수 백오프 + 지터
    @Retryable(
            includes = MessageDeliveryException.class,
// 원인(cause) 예외까지 매칭한다
            maxRetries = 4,
            delay = 100,          // 첫 대기 100ms
            multiplier = 2,       // 100 → 200 → 400 →
800ms
            jitter = 10,          // 대기 시간에 ±10ms 랜덤 오
차
            maxDelay = 1000)      // 대기 시간 상한
    public void sendWithBackoff() { ... }
```

```
    // 리액티브 메소드도 지원
    @Retryable(maxRetries = 4, delay = 100)
    public Mono<Void> sendReactive() { ... }
}
```

`@Retryable(MessageDeliveryException.class)`는 `@Retryable(includes = MessageDeliveryException.class)`의 축약형이다. `maxRetriesString`, `delayString`처럼 `String` 버전 속성도 있어서 `${...}` 플레이스홀더나 SpEL로 설정 파일 값을 주입할 수 있다.

> **지터(jitter)는 왜 필요할까?** 여러 클라이언트가 동시에 실패하면 똑같은 간격으로 동시에 재시도해서 서버를 다시 몰아친다. 대기 시간에 랜덤 오차를 섞으면 재시도 시점이 흩어진다.

0.2 RetryTemplate

- **프로그래밍 방식**이며 재시도 단위가 **코드 블록**이다.
- 빈이 아니어도 어디서나 `new`로 생성할 수 있다. 인스턴스가 매우 가벼워서 호출마다 다른 정책으로 즉석에서 만들어 써도 된다.
- 재시도가 소진되면 `invoke()`는 **마지막 원본 예외**를 던지고, `execute()`는 모든 시도의 결과를 담은 **checked 예외** `RetryException` 을 던진다.
- `RetryOperations` 인터페이스를 구현하므로 테스트에서 mock/stub으로 바꿔 끼울 수 있다.

```
// 기본 정책 (RetryPolicy.withDefaults())
var retryTemplate = new RetryTemplate();

// 재시도 횟수만 바꿀 때
```

```
var retryTemplate = new
RetryTemplate(RetryPolicy.withMaxRetries(4));

// 세부 정책은 빌더로
var retryPolicy = RetryPolicy.builder()
        .includes(MessageDeliveryException.class)
        .maxRetries(4)
        .delay(Duration.ofMillis(100))
        .multiplier(2)
        .jitter(Duration.ofMillis(10))
        .maxDelay(Duration.ofSeconds(1))
        .build();
var retryTemplate = new RetryTemplate(retryPolicy);
```

`invoke()` vs `execute()`

```
// invoke : @Retryable과 같은 방식. 마지막 원본(unchecked) 예
외를 그대로 받는다
try {
    String result = retryTemplate.invoke(() ->
client.call());
} catch (MessageDeliveryException ex) {
    // 원본 메소드에서 발생한 예외
}

// execute : 모든 시도의 예외를 담은 RetryException(checked)을
받는다
try {
    String result = retryTemplate.execute(() ->
client.call());
} catch (RetryException ex) {
    ex.getExceptions();     // 시도별 예외 전체
    ex.getLastException();  // 마지막 예외
}
```

`invoke()`는 마지막 예외 말고는 재시도 상태를 알려주지 않는다. 중간 과정을 보고 싶으면 `RetryListener`를 등록한다. 재시도 전후, 매 시도, 소진·인터럽트·타임아웃 같은 결과를 받을 수 있고, 여러 리스너는 `CompositeRetryListener`로 묶을 수 있다.

0.3 언제 무엇을 써야 할까?

@Retryable이 맞는 경우

- 메소드 하나가 **다시 해도 괜찮은 단위**일 때
- 정책이 고정되어 있거나, 설정 파일로 빼는 것으로 충분할 때
- 리액티브 메소드(`Mono` / `Flux`)를 사용할 때
- 비즈니스 코드에 재시도 로직을 섞고 싶지 않을 때

RetryTemplate이 맞는 경우

- **메소드의 일부만** 재시도해야 할 때 : `@Retryable`은 메소드를 처음부터 다시 호출하지만, RetryTemplate은 일부 로직만 감싸면 된다.
- **프록시를 탈 수 없을 때** : Spring Bean이 아니거나, 같은 클래스 안에서 호출하는 경우다.
- **호출마다 정책이 달라야 할 때** : 요청 종류나 런타임 값에 따라 재시도 횟수·간격을 바꿔야 하는 경우다.
- **재시도 과정 전체를 다뤄야 할 때** : `execute()`의 `RetryException`으로 모든 시도의 예외를 받거나, `RetryListener`로 각 단계에 반응해야 하는 경우다.

```
public void placeOrder(Order order) {
    validate(order);            // 재시도할 필요 없는 로직
    saveOrder(order);           // 재시도하면 중복 저장되는 로직
```

```
    // 외부 결제 API 호출만 재시도한다
    retryTemplate.invoke(() ->
paymentClient.pay(order));
}
```

`@Retryable`을 `placeOrder`에 붙였다면 실패할 때마다 `saveOrder`까지 다시 실행된다.

0.4 공통 사항

- **재시도는 멱등한 작업에만 해야 한다.** 즉, 같은 요청이 두 번 처리되어도 안전해야 한다. 타임아웃으로 실패한 요청은 실제로는 서버에서 처리됐을 수도 있다. 결제처럼 멱등하지 않은 작업은 멱등성 키(idempotency key)를 함께 보내는 식으로 보완해야 한다.
- **재시도할 예외를 좁혀야 한다.** 기본값은 모든 예외를 재시도하므로, 검증 실패처럼 다시 해도 똑같이 실패하는 예외까지 재시도한다. 네트워크 오류나 일시적인 DB 오류 같은 **일시적(transient) 장애**만 `includes`로 지정하는 것이 좋다.
- **이름 충돌** : `@Retryable` 어노테이션(`org.springframework.resilience.annotation.Retryable`)과 RetryTemplate에 넘기는 함수형 인터페이스 `Retryable`(`org.springframework.core.retry.Retryable`)은 **단순 이름이 같다.** 그래서 한 파일에서 둘 다 import하면 컴파일 오류가 난다. 둘 다 써야 하면 한쪽은 패키지를 포함한 풀네임으로 적어야 한다. (RetryTemplate에 람다만 넘긴다면 `Retryable` 인터페이스를 import할 일이 없어서 충돌하지 않는다)

```
import
org.springframework.resilience.annotation.Retryable;
```

```
@Retryable
public void send() { ... }

// RetryListener 구현 등에서 인터페이스가 필요하면 풀네임으로
public void onRetryableExecution(RetryPolicy policy,

org.springframework.core.retry.Retryable<?> retryable,
                                 RetryState state) {
... }
```

다른 프록시 기반 어노테이션과 같이 쓸 때

같은 메소드에 여러 어노테이션이 붙으면 AOP가 정해진 순서로 인터셉터를 감싼다. 이 순서가 재시도 동작을 결정한다.

조합	어드바이스 체인	동작
`@Transactional`	Retry(바깥) → Transaction(안쪽) → 메소드	**재시도 한 번마다 새 트랜잭션이 열린다.** 실패하면 롤백된 뒤 재시도한다
`@Cacheable`	Retry(바깥) → Cache(안쪽) → 메소드	매 시도마다 캐시를 먼저 확인한다. 그사이 캐시가 채워지면 메소드를 호출하지 않고 캐시 값을 반환한다
`@Async`	Async(바깥) → Retry(안쪽) → 메소드	비동기 스레드에 한 번만 제출되고, 모든 재시도가 그 스레드에서 일어난다. 호출 스레드는 대기 시간 동안 막히지 않는다

`@Transactional`과의 순서는 재시도가 트랜잭션 **바깥**을 감싸는 구조로 고정되어 있으며, `@EnableResilientMethods`의 `order` 속성으로도 바꿀 수 없다. 데드락 같은 일시적인 DB 오류를 재시도할 때 원하는 동작이 바로 이것이다. 만약 순서가 반대였다면 이미 롤백 마킹된 트랜잭션 안에서 재시도하게 되어 의미가 없다.

```
@Transactional
@Retryable(TransientDataAccessException.class)
public void updateRecord() {
```

```
    // 각 재시도가 독립된 트랜잭션에서 실행된다
}
```

`@Async`와의 순서는 `@EnableResilientMethods`와 `@EnableAsync`의 `order` 속성으로 바꿀 수 있다. 하지만 순서를 뒤집으면 재시도 인터셉터는 즉시 반환되는 `Future`만 보게 되어, 비동기 실행 중 발생한 예외를 재시도하지 못한다. 실무에서는 기본 순서를 그대로 쓰는 것이 맞다.